

C/C++ Pointers, References, and Memory Management

1. Overview

Pointers store memory addresses; references provide aliases to existing objects; memory management controls when storage is acquired, used, and released. These ideas matter in GPU programming because host and device memory are distinct address spaces, buffers are passed through pointer-like handles, and performance depends heavily on where data lives and how it is accessed.

They appear in operating systems, allocators, containers, device drivers, numerical libraries, CUDA/HIP kernels, and almost every performance-sensitive C++ system. Interviewers use them to test whether you understand object lifetime, ownership, aliasing, undefined behavior, and low-level performance—not merely pointer syntax.

2. Core Idea

Think of memory as numbered storage boxes. A value occupies a box; its address is the box number. A pointer holds that number. A reference is another name attached to the same box.

```
int value = 7;
int* p = &value;    // p stores value's address
int& r = value;     // r aliases value

*p = 9;             // dereference p
r += 1;             // modifies the same object
// value is now 10
```

Step by step: `value` gets storage; `&value` obtains its address; `p` stores it; `*p` follows the address; `r` directly names the same object. None of these creates another `int`.

On a GPU, the same mental model applies, but the pointed-to storage may be host, device, shared, local, constant, or unified memory. A pointer is meaningful only in an address space where it is valid.

3. Important Subtopics

Pointer syntax and pointer arithmetic

`T*` points to `T`; `&x` takes an address; `*p` dereferences it. `p + i` advances by `i * sizeof(T)` bytes. This makes arrays efficient, but an out-of-bounds pointer or dereference causes undefined behavior. Interviewers often ask why `p + 1` does not advance by one byte.

```
int a[] = {10, 20, 30};
int* p = a;
int x = *(p + 2); // 30
```

Null, dangling, wild, and void pointers

- `nullptr` means “points to no object” and should not be dereferenced.
- A dangling pointer refers to an object whose lifetime ended.
- An uninitialized, or wild, pointer has an indeterminate value.
- `void*` can hold an object address but must be converted before typed dereference.

The interview angle is identifying which states are safe to copy, compare, or dereference.

References

An lvalue reference (`T&`) normally binds to an existing named object; an rvalue reference (`T&&`) can bind to a temporary and enables move semantics. References cannot normally be reseated and must be initialized. A `const T&` avoids a copy while preventing modification through that reference.

Stack, heap, static, and GPU storage

Automatic objects typically live for a block invocation; dynamic objects live until released; static objects live for the program; thread-local objects live for a thread. “Stack versus heap” is an implementation-oriented shorthand—the C++ standard specifies storage duration and lifetime.

GPU code adds global/device memory, per-block shared memory, per-thread registers/local memory, and constant memory. Their capacities, latency, and visibility differ.

Ownership and RAI

Ownership answers who must release a resource. RAI binds a resource to an object whose destructor releases it. Prefer values and standard containers; use `std::unique_ptr` for exclusive ownership and `std::shared_ptr` only for genuinely shared lifetime.

```
auto data = std::make_unique<float[]>(n); // automatically released
```

Interviewers expect “Rule of Zero”: if members manage themselves, avoid writing custom destructors/copy/move operations.

Allocation and deallocation

`new` pairs with `delete`; `new[]` pairs with `delete[]`. `malloc/free` allocate raw bytes and do not run constructors/destructors. Mixing families is undefined behavior. In CUDA, common pairs include `cudaMalloc/cudaFree` and `cudaHostAlloc/cudaFreeHost`.

Copying and moving

A shallow copy duplicates pointer values; a deep copy duplicates owned data. Move operations transfer resources and leave the source valid but in an unspecified state. Raw owning pointers make these rules error-prone, which is why RAI types are preferred.

const correctness and aliasing

`const int* p` points to a read-only `int`; `int* const p` is a non-reseatable pointer; `const int* const p` is both. Aliasing means multiple expressions may access the same storage. It affects correctness and compiler optimization; CUDA/C++ code may use `__restrict__` only when the no-alias promise is true.

4. Real-World Example

A host program allocates two input arrays and one output array, allocates matching GPU buffers, copies inputs to the device, launches a kernel, copies the result back, and releases device storage. Every step involves ownership, lifetimes, and valid address spaces. A kernel must not use a freed device pointer, and asynchronous copies require buffers to remain alive until the operation completes.

RAII wrappers around device allocations are common because early returns and exceptions otherwise leak memory.

5. Diagrams / Mental Models

```

Host address space                                GPU address space
+-----+                                         +-----+
| vector A | --copy H->D-----> | device buffer A|
+-----+                                         +-----+
      ^                                           ^
    host pointer                             device-valid pointer

Lifetime: allocate -> initialize -> use -> synchronize -> release

```

Form	Meaning
<code>T x</code>	object/value
<code>T* p = &x</code>	pointer holding <code>x</code> 's address
<code>*p</code>	object reached through <code>p</code>
<code>T& r = x</code>	alias for <code>x</code>
<code>const T* p</code>	pointer to const <code>T</code>
<code>T* const p</code>	const pointer to mutable <code>T</code>

6. Common Interview Questions

1. **Pointer vs reference?** A pointer is an object storing an address; it can be null and reseated. A reference aliases an object and must be initialized. **Expected:** syntax, nullability, reseating, use cases. **Mistake:** claiming references are always implemented without storage.

2. **What is a dangling pointer?** A pointer to storage whose object lifetime ended, such as a returned local address. **Expected:** lifetime and prevention via RAI. **Mistake:** saying setting one copied pointer to null fixes other aliases.
3. **What is a memory leak?** Allocated resource becomes unreachable without being released. **Expected:** ownership and RAI. **Mistake:** confusing a leak with temporary high memory usage.
4. **new/delete vs malloc/free?** C++ allocation constructs/destructs typed objects; C allocation manages raw bytes. **Expected:** correct pairing. **Mistake:** mixing them.
5. **Shallow vs deep copy?** Shallow copy duplicates handles; deep copy duplicates owned resources. **Expected:** double-free/alias consequences. **Mistake:** assuming all pointer copies need deep copying.
6. **What is RAI?** Acquire a resource during object construction and release it in the destructor. **Expected:** exception safety and deterministic cleanup. **Mistake:** describing it as garbage collection.
7. **unique_ptr vs shared_ptr?** Exclusive movable ownership versus reference-counted shared ownership. **Expected:** `weak_ptr` for non-owning cycle breaking. **Mistake:** defaulting to `shared_ptr` everywhere.
8. **What is undefined behavior?** Behavior for which the C++ standard imposes no requirements, such as use-after-free or out-of-bounds access. **Expected:** compiler may assume it never happens. **Mistake:** saying it always crashes.
9. **Why can pointer arithmetic be dangerous?** It is valid only within an array object (plus one-past for comparison, not dereference). **Expected:** scaling by element size. **Mistake:** treating arbitrary addresses as portable arrays.
10. **Why pass by `const&`?** Avoid copying a large object while promising not to mutate through the parameter. **Expected:** small scalars are usually better by value. **Mistake:** assuming it prevents all external mutation.
11. **What is alignment?** A requirement/preference that an object address be a multiple of a boundary. **Expected:** correctness on some hardware and transaction efficiency. **Mistake:** confusing alignment with size.
12. **Why synchronize before freeing an async GPU buffer?** Kernel launches/copies may still be using it. **Expected:** stream ordering and lifetime. **Mistake:** assuming launch return means completion.

7. Deep-Dive Questions

1. **How does strict aliasing affect optimization?** Compilers may assume pointers of unrelated types do not address the same object, enabling reordering/vectorization. Violating permitted aliasing rules is undefined; use `std::memcpy`/`std::bit_cast` for representation transfer.
2. **Why can `shared_ptr` leak?** Reference cycles keep counts nonzero. Make back-edges non-owning with `std::weak_ptr` or redesign ownership.
3. **What is placement `new`?** It constructs an object in supplied storage; the caller controls storage and must arrange destruction. It is useful in allocators but demands alignment and lifetime care.
4. **What is unified memory?** A managed virtual address range accessible by CPU and GPU, with pages migrated or mapped as needed. It simplifies addressing but page faults and migration can hurt performance.
5. **Why does `restrict` help kernels?** A valid no-alias guarantee lets the compiler cache values and reorder loads/stores. If pointers alias despite the promise, results are undefined.

8. Comparison Tables

Feature	Pointer	Reference
Can be null	Yes	Not when validly bound
Can be reused	Yes	No
Requires dereference syntax	Yes	No
Supports arithmetic	Yes	No

Typical use	Optional object, arrays, low-level APIs	Aliasing/pass-by-reference
-------------	---	----------------------------

Tool	Ownership	Overhead	Best use
Raw pointer	Usually non-owning	None	Observation, traversal, C API
<code>unique_ptr</code>	Exclusive	Essentially raw-pointer-sized	Default dynamic ownership
<code>shared_ptr</code>	Shared	Control block, atomic ref-counting	Truly shared lifetime

<code>weak_ptr</code>	Non-owning	Control-block access	Observe shared object/break cycles
-----------------------	------------	----------------------	------------------------------------

Memory	Scope	Typical GPU role
Registers	One thread	Fast scalar working data
Shared memory	One block	Explicitly managed reuse/cooperation
Global memory	Device/grid	Large input and output buffers
Constant memory	Device, read-only to kernels	Small broadcast constants
Host pinned memory	Host; DMA-friendly	Faster/asynchronous transfers

9. Common Mistakes

- Returning pointers or references to local variables.
- Dereferencing null, dangling, one-past-end, or device-invalid pointers.
- Mismatching `new[]` with `delete`, or allocation families.
- Owning the same raw pointer in multiple objects.
- Using `shared_ptr` to avoid deciding ownership.
- Forgetting virtual destructors when deleting polymorphically through a base pointer.
- Assuming a GPU kernel is complete immediately after launch.
- Treating `volatile` as thread synchronization.

10. Edge Cases / Special Cases

- A one-past-the-end pointer may be formed and compared but not dereferenced.
- Deleting `nullptr` is safe.
- An empty container's `data()` need not be dereferenceable.
- Reallocation invalidates pointers, references, and iterators into a `std::vector`.
- `const_cast` does not make an originally const object safely writable.

- Pinned host memory improves transfer capability but is scarce and expensive to allocate.
- Device pointers may look like ordinary addresses under unified virtual addressing but accessibility and synchronization rules still matter.

11. How to Explain in Interview

"A pointer is a value containing an address, while a reference is an alias to an existing object. Correct low-level C++ depends on knowing ownership and lifetime: I prefer values and RAII types, use raw pointers mainly as non-owning views, and make synchronization explicit when CPU or GPU operations can outlive a buffer."

12. Quick Revision Notes

- Address-of: `&x`; dereference: `*p`; member through pointer: `p->m`.
- `nullptr` is the C++ null pointer literal.
- Lifetime is not the same as storage allocation.
- Rule of Zero beats handwritten copy/move/destructor code.
- `unique_ptr` first; `shared_ptr` only for shared ownership.
- Trap: async GPU work extends required buffer lifetime.
- Trap: pointer arithmetic is scaled and array-bounded.

13. Practice Tasks

1. Swap two integers using pointers and then references.
2. Implement a small RAII wrapper for a `cudaMalloc` allocation if CUDA is available.
3. Find and fix a use-after-free with AddressSanitizer.
4. Demonstrate `std::vector` pointer invalidation after growth.
5. Explain ownership in a linked list and implement it with `unique_ptr`.
6. Trace host/device buffer lifetimes for an asynchronous kernel and copy.

14. Final Cheat Sheet

Item	Recall
Core definition	Pointer stores an address; reference aliases an object; ownership governs cleanup
Why it matters	Safety, lifetime, performance, GPU buffer handling
Most asked	Dangling pointer, RAII, smart pointers, shallow/deep copy, <code>const</code>
Main comparison	Raw pointer vs reference vs smart pointer
One-line answer	"Use pointers for address-based access, references for aliases, and RAII for ownership."

Arrays, Structs, and Templates

1. Overview

An array is a contiguous sequence of same-type elements. A struct groups related fields into one type. A template describes code parameterized by types or values. Together they let C++ represent data compactly

and write reusable, zero-overhead algorithms—central concerns in GPU kernels.

Real systems use arrays for buffers, images, tensors, and packet batches; structs for records and hardware-facing layouts; templates for containers, numerical kernels, and compile-time specialization. Interviewers ask about layout, decay, copying, padding, generic programming, and performance tradeoffs.

2. Core Idea

Imagine a warehouse: an array is a row of identical bins, a struct is one labeled package containing different items, and a template is a blueprint that can produce packages or algorithms for different types.

```
template<class T>
struct Point { T x, y; };

Point<float> points[2]{{1, 2}, {3, 4}};
float y = points[1].y; // 4
```

The compiler instantiates `Point<float>`, lays two objects contiguously, computes the second object's offset, then selects its `y` member. GPU programming uses the same mechanisms for arrays of particles, pixels, or matrix elements.

3. Important Subtopics

Built-in arrays and `std::array`

`T a[N]` has fixed compile-time extent and contiguous elements. In most expressions it decays to `T*`, losing size information. `std::array<T, N>` remains a value type with `.size()`, iterators, and safe copying. Interviewers test array decay and `sizeof` behavior.

Dynamic contiguous storage

`std::vector<T>` owns a growable contiguous allocation. Growth can reallocate and invalidate iterators/pointers. `std::span<T>` is a non-owning view of contiguous elements and carries a length, making APIs safer than `(pointer, size)` pairs.

Multidimensional arrays and indexing

C/C++ built-in multidimensional arrays are row-major: the last index varies fastest. For a flat row-major matrix, element `(r,c)` is `data[r * cols + c]`. This formula is vital for correct GPU thread-to-data mapping.

Struct layout, alignment, and padding

Members appear in declaration order but padding may be inserted to satisfy alignment. Layout affects ABI, file/network formats, and GPU transaction efficiency.

```
struct S { char tag; int value; }; // often 8 bytes, not 5
```

Do not serialize a general struct by dumping raw bytes: padding, endianness, and versioning differ.

Array of Structures vs Structure of Arrays

AoS stores complete records consecutively; SoA stores each field in its own array. AoS is convenient when each operation needs every field. SoA often gives better SIMD/GPU coalescing when many lanes access one field.

Function and class templates

Function templates generate functions for deduced or explicit types; class templates generate types. Instantiation happens when needed. Interviewers expect awareness that definitions generally belong in headers because the compiler must see them.

Specialization and constraints

Specialization customizes a template for particular arguments. C++20 concepts constrain valid inputs and improve errors. Prefer ordinary overloads when they express the behavior clearly; specialize only for a genuine type-specific implementation.

Compile-time value parameters

Non-type template parameters such as tile size let compilers unroll and allocate fixed resources.

```
template<int Tile>
__global__ void matmul(const float* a, const float* b, float* c);
```

The tradeoff is more compiled variants and code size.

4. Real-World Example

A particle simulation can store `struct Particle { float x,y,z,vx,vy,vz; };` in an AoS. If a GPU kernel updates only positions, loading interleaved velocity fields wastes bandwidth. Converting to SoA—six separate arrays—lets adjacent threads read adjacent `x` values, producing coalesced memory transactions. A templated kernel can support `float` and `double` or fixed block sizes without runtime branches.

5. Diagrams / Mental Models

```
AoS: [x y z][x y z][x y z][x y z]
      thread0 thread1 thread2 thread3

SoA: [x x x x][y y y y][z z z z]
      adjacent threads read adjacent x values

Row-major 2 x 3 matrix:
logical:  [a b c]      memory: [a b c d e f]
          [d e f]      index(r,c) = r*3+c
```

6. Common Interview Questions

1. **Array vs pointer?** An array owns/denotes a fixed sequence; a pointer stores an address. Arrays often decay to pointers but are not pointers. **Expected:** size and assignment differences. **Mistake:** saying they are identical.
2. **Why are arrays cache-friendly?** Contiguous elements provide spatial locality and predictable prefetching. **Expected:** cache lines. **Mistake:** claiming every access is constant latency.
3. **`std::array` vs `std::vector`?** Fixed inline extent versus dynamically sized owned storage. **Expected:** both contiguous. **Mistake:** assuming vector elements are individually heap allocated.
4. **What is struct padding?** Unused bytes inserted for member alignment. **Expected:** order can change size. **Mistake:** assuming sizes always sum.
5. **AoS vs SoA?** Record-major convenience versus field-major access efficiency. **Expected:** workload-dependent answer. **Mistake:** declaring one universally faster.
6. **How is a 2D flat array indexed?** Row-major $(r, c) \rightarrow r * \text{cols} + c$. **Expected:** bounds and stride. **Mistake:** multiplying by rows.
7. **What is array decay?** In many expressions, an array converts to a pointer to its first element. **Expected:** exceptions include `sizeof`, `decltype`, and address-of. **Mistake:** saying size travels with the pointer.
8. **What is a template?** A compile-time pattern for generating types/functions from arguments. **Expected:** static polymorphism. **Mistake:** calling it a textual macro.
9. **Why define templates in headers?** The definition must generally be visible at the point of implicit instantiation. **Expected:** explicit instantiation is an alternative. **Mistake:** saying templates can never be separated.
10. **Overloading vs specialization?** Overloads participate in overload resolution; specialization customizes a primary template. **Expected:** prefer clear overloads for functions. **Mistake:** partially specializing function templates—C++ does not allow it.
11. **What is `std::span`?** A lightweight non-owning contiguous range view. **Expected:** no lifetime extension. **Mistake:** treating it as owning memory.
12. **Why template GPU tile sizes?** Compile-time constants enable unrolling and fixed shared-memory shapes. **Expected:** code-size/compile-time tradeoff. **Mistake:** generating arbitrary variants without evidence.

7. Deep-Dive Questions

1. **When is a struct safe for binary interchange?** Only under a deliberately specified representation: fixed-width fields, byte order, packing/alignment, and versioning. Trivially copyable alone does not establish cross-platform format compatibility.
2. **How does alignment affect GPU access?** Naturally aligned, adjacent accesses are easier to combine into fewer memory transactions; misalignment may split transactions.
3. **What is template code bloat?** Each distinct instantiation may emit machine code. Reduce variants, move type-independent work out, or explicitly instantiate measured hot combinations.
4. **What is SFINAE, and how do concepts improve it?** Substitution failures can remove candidates instead of producing hard errors. Concepts state requirements directly and usually yield clearer diagnostics.
5. **Can a multidimensional allocation be non-contiguous?** Yes: an array of row pointers may reference separate allocations. It differs from one flat allocation in locality, transfer convenience, and indexing metadata.

8. Comparison Tables

Feature		Built-in array	<code>std::array</code>	<code>std::vector</code>	<code>std::span</code>
Owns elements		Yes	Yes	Yes	No
Extent		Compile time	Compile time	Runtime	Runtime or static
Contiguous		Yes	Yes	Yes	Views contiguous data
Copy assignable as a unit		No	Yes	Yes	Copies view only
Layout	Strength	Weakness		GPU fit	
AoS	Natural record operations	Field access may waste bandwidth		Good when threads consume full records	
SoA	Excellent field-wise locality	More arrays, less ergonomic		Good for coalesced same-field access	
AoSoA	Balances blocks and fields	More indexing complexity		Useful when tuning for vector/warp width	
Mechanism	Binding time	Main use			
Template	Compile time	Generic zero-overhead code			
Virtual function	Runtime	Open-ended subtype dispatch			
Macro	Preprocessing	Conditional compilation; avoid for typed logic			

9. Common Mistakes

- Using `sizeof(pointer)` to infer the original array length.
- Returning a `span` to destroyed storage.
- Keeping pointers into a vector across reallocation.
- Assuming struct layout has no padding.
- Copying C arrays with assignment.
- Confusing a 2D pointer-of-pointers with contiguous matrix storage.
- Choosing AoS/SoA by fashion instead of access pattern.
- Creating excessive template variants that increase build time and binary size.

10. Edge Cases / Special Cases

- Zero-length built-in arrays are not standard C++.
- `std::array<T,0>` is valid, but `front()/back()` are invalid.
- `vector<bool>` is a packed specialization; its element access is a proxy, not `bool&`.
- Flexible array members are a C extension/pattern, not standard C++ structs.
- Padding bytes can contain indeterminate values and make raw `memcmp` misleading.
- `alignas` can strengthen alignment but cannot request weaker alignment than required.
- Template recursion/instantiation has compiler limits and can produce poor diagnostics.

11. How to Explain in Interview

“Arrays give contiguous homogeneous storage, structs define heterogeneous records, and templates generate type- or value-specific code at compile time. In GPU work I choose the data layout from the access pattern—often SoA for coalescing—and use templates selectively for types or tile sizes that benefit from compile-time specialization.”

12. Quick Revision Notes

- Arrays are contiguous; pointers do not carry length.
- Row-major index: `row * columns + column`.
- Struct size includes possible padding and tail padding.
- `vector` growth invalidates views into its storage.
- `span` observes; it does not own.
- AoS favors whole-record use; SoA favors same-field parallel use.
- Templates are typed compile-time generation, not macros.

13. Practice Tasks

1. Flatten and reconstruct indices for a 3D tensor.
2. Print `sizeof`, `alignof`, and member offsets for several struct orders.
3. Benchmark summing `x` fields in AoS and SoA layouts.
4. Write a templated vector-add function for `int`, `float`, and `double`.
5. Replace a `(pointer, length)` API with `std::span` and discuss lifetime.
6. Write a tiled matrix-kernel signature parameterized by tile size.

14. Final Cheat Sheet

Item	Recall
Core definition	Array = contiguous sequence; struct = record; template = compile-time pattern
Why it matters	Data layout and specialization directly affect GPU bandwidth and execution
Most asked	Array decay, padding, AoS/SoA, vector invalidation, templates
Main comparison	Array vs pointer; AoS vs SoA; templates vs runtime polymorphism
One-line answer	“Lay out data for the access pattern, preserve bounds, and specialize only useful variants.”

Bit Operations

1. Overview

Bit operations manipulate the individual binary digits of integer values using AND, OR, XOR, NOT, shifts, masks, and bit-counting operations. They matter because hardware flags, permissions, encodings, hashes, graphics formats, and GPU lane logic are naturally expressed as bits.

Interviewers ask bit questions to test binary reasoning, signedness awareness, edge-case discipline, and the ability to replace an apparently complex task with a precise constant-time operation.

2. Core Idea

Imagine switches arranged in a row. A mask selects switches: AND keeps selected on-bits, OR turns chosen bits on, XOR toggles them, and NOT flips them.

```
unsigned flags = 0;
constexpr unsigned Ready = 1u << 2;
flags |= Ready;           // set
bool ready = flags & Ready;
flags &= ~Ready;          // clear
```

For `Ready = 00000100`, OR sets bit 2, AND tests it, and AND with the complemented mask clears it. Using unsigned types makes shifting and bit patterns easier to reason about.

3. Important Subtopics

Binary and hexadecimal representation

One hex digit represents four bits, so hex is compact and aligns with bit boundaries: `0xF0 == 111100002`. Interviews commonly require converting, masking, and recognizing powers of two.

AND, OR, XOR, and NOT

- `x & mask`: retain bits set in both.
- `x | mask`: set selected bits.
- `x ^ mask`: toggle/detect differing bits.
- `~x`: invert every bit in the promoted integer type.

XOR satisfies $x \oplus x = 0$ and $x \oplus 0 = x$, useful for parity and cancellation—but not a license to replace readable swaps.

Shifts

`x << k` shifts left and `x >> k` shifts right. For unsigned values and valid counts, left shift corresponds to multiplication modulo the width, while right shift is floor division by powers of two. Shift counts at least the type width are invalid/undefined in C++.

Masks and bit fields

A mask isolates a field. To extract `width` bits starting at `offset`:

```
auto field = (value >> offset) & ((1u << width) - 1);
```

Production code must handle `width == type_width` separately to avoid an invalid shift.

Two's complement and signed integers

Modern C++ specifies two's-complement representation, but signed overflow is still undefined. Prefer unsigned types for intentional modular bit arithmetic. $-x$ in two's complement is invert-plus-one.

Power-of-two techniques

For unsigned $x > 0$, $x \& (x - 1)$ clears the lowest set bit; therefore $(x \& (x - 1)) == 0$ detects a power of two. The $x > 0$ guard is essential.

Population count and bit scanning

C++20 `<bit>` includes `std::popcount`, `std::countl_zero`, `std::countr_zero`, `std::has_single_bit`, rotations, and endian helpers. These map well to hardware and are clearer than handwritten loops.

GPU-specific voting and lane masks

GPU warp/wavefront vote intrinsics produce bit masks representing which lanes satisfy a predicate. Ballot masks support compaction, branch coordination, and prefix-like operations. Mask width and active-lane semantics depend on the platform.

4. Real-World Example

A GPU kernel processes 32 predicates per warp. A ballot instruction packs their truth values into a 32-bit mask. `popcount(mask)` gives how many lanes matched; counting set bits below a lane gives its compacted output position. This avoids one atomic operation per matching thread.

Bit masks also encode pixel channels, memory permissions, network headers, and allocator free lists.

5. Diagrams / Mental Models

```
x          = 10110100
mask       = 00111100
x & mask   = 00110100   isolate selected region
x | mask   = 10111100   set selected region
x ^ mask   = 10001000   toggle selected region

x          = 10110000
x - 1      = 10101111
x & (x-1)  = 10100000   lowest set bit cleared
```

Task	Expression for unsigned x
Test bit k	$(x \& (1u \ll k)) \neq 0$
Set bit k	$x \mid 1u \ll k$
Clear bit k	$x \&= \sim(1u \ll k)$
Toggle bit k	$x \wedge= 1u \ll k$

Task	Expression for unsigned x
Lowest set bit value	$x \& -x$ with unsigned modular arithmetic
Clear lowest set bit	$x \&= x - 1$

6. Common Interview Questions

1. **How do you test bit k ?** AND with $1u \ll k$ and compare against zero. **Expected:** valid range for k . **Mistake:** comparing result to 1 instead of nonzero.
2. **Set, clear, toggle a bit?** OR, AND with complemented mask, XOR. **Expected:** unsigned mask. **Mistake:** using logical operators $\&\&/||$.
3. **Check power of two?** $x > 0 \&\& (x \& (x-1)) == 0$. **Expected:** zero guard. **Mistake:** accepting zero.
4. **Count set bits?** Use `std::popcount`; classic loop repeatedly clears the lowest set bit. **Expected:** complexity proportional to set bits for the loop. **Mistake:** reinventing when `<bit>` is available.
5. **Find the unique value when others occur twice?** XOR all values. **Expected:** cancellation proof. **Mistake:** applying it when duplicates occur an arbitrary number of times.
6. **What does $x \& -x$ do?** Isolates the lowest set bit for unsigned/two's-complement arithmetic. **Expected:** representation and zero behavior. **Mistake:** ignoring signed overflow at minimum signed value.
7. **Arithmetic vs logical right shift?** Arithmetic replicates sign; logical inserts zeros. Unsigned right shift is logical; signed negative right shift is defined as arithmetic in current C++. **Expected:** signedness. **Mistake:** generalizing across languages/old standards.
8. **Why use hex for masks?** Each digit maps to four bits, making fields visible. **Expected:** readability. **Mistake:** claiming hex changes stored representation.
9. **Can left shift replace multiplication?** Sometimes for unsigned powers of two, but multiplication is clearer and compilers optimize it. **Expected:** overflow/shift-count rules. **Mistake:** manual "optimization."
10. **How do you extract a bit field?** Shift it to bit zero, then mask its width. **Expected:** width boundary cases. **Mistake:** masking before/after with wrong offset.
11. **What is endianness?** Byte order in multi-byte object representation, not the order of bits in a byte. **Expected:** big vs little endian. **Mistake:** reversing displayed binary digits.
12. **What is a GPU ballot?** A collective that returns a lane-bit mask for a predicate. **Expected:** active mask and warp scope. **Mistake:** assuming inactive lanes contribute reliable bits.

7. Deep-Dive Questions

1. **Why is signed overflow dangerous to bit algorithms?** It is undefined, allowing optimizers to remove paths based on "overflow cannot happen." Use unsigned arithmetic for modular intent.
2. **How can a ballot compact results?** Ballot matching lanes, compute each matching lane's rank by popcounting lower active bits, reserve a block-sized output range, then write at base plus rank.
3. **How do you avoid undefined full-width masks?** Use a wider type, branch on full width, or standard helpers. $(1u \ll 32)$ is invalid for a 32-bit unsigned value.
4. **How are floating-point bits inspected safely?** Use `std::bit_cast` to an equal-sized unsigned integer, not pointer type-punning that violates aliasing.
5. **Why can bitfields in C++ structs be unsuitable for protocols?** Allocation order, packing, and alignment are implementation-defined; explicit masks and shifts over fixed-width integers are portable.

8. Comparison Tables

Operator	Bitwise	Logical counterpart	Difference
<code>&</code>	Per-bit AND	<code>&&</code>	Logical form short-circuits and yields <code>bool</code>
<code> </code>	<code> </code>	Per-bit OR	<code> </code>
<code>^</code>	Per-bit XOR	none	True where bits differ
<code>~</code>	Per-bit NOT	<code>!</code>	Logical NOT converts truth value
Type choice	Shift behavior	Overflow	Recommendation
Unsigned integer	Predictable logical right shift	Modular	Preferred for masks
Signed integer	Sign-aware; negative values need care	Undefined	Use for numeric values, not raw masks

9. Common Mistakes

- Confusing bitwise and logical operators.
- Forgetting operator precedence; use parentheses around masks/tests.
- Shifting by a negative count or by at least the type width.
- Using signed `1 << 31` and triggering undefined behavior.
- Accepting zero as a power of two.
- Assuming endianness changes arithmetic bit positions.
- Writing clever XOR tricks when `std::swap` or `<bit>` is clearer.
- Ignoring inactive lanes in GPU mask operations.

10. Edge Cases / Special Cases

- Integer promotions can make `~uint8_t{0}` an `int`, not an 8-bit value.
- `std::countr_zero(0)` is defined by the standard helper, while many hardware intrinsics historically had special rules.
- Rotation differs from shifting because discarded bits wrap around; use `std::rotrl/std::rotr`.
- A mask literal needs a wide enough suffix, such as `1ull << k`.
- Atomic bitwise operations may be needed when multiple threads update the same flags.
- GPU warp size is not universally 32; write to the target platform's execution model.

11. How to Explain in Interview

"Bit operations treat an integer as a fixed-width set of binary flags. Masks let me test, set, clear, toggle, or extract fields efficiently. I use unsigned fixed-width types, validate shift counts, prefer C++ `<bit>` helpers, and account for active-lane masks in GPU collectives."

12. Quick Revision Notes

- AND selects, OR sets, XOR toggles, NOT flips.
- Use unsigned/fixed-width types for bit patterns.

- `x & (x-1)` clears the lowest set bit.
- Power of two needs `x != 0`.
- Shift count must be within width.
- Endianness is byte order.
- Prefer `<bit>` over handwritten tricks.

13. Practice Tasks

1. Implement set/clear/test/toggle helpers with boundary checks.
2. Pack and unpack three fixed-width fields in a `uint32_t`.
3. Solve single-number, subset enumeration, and power-of-two problems.
4. Compare a manual popcount loop with `std::popcount`.
5. Use `std::bit_cast` to print a float's sign, exponent, and fraction.
6. On a GPU platform, use ballot and popcount to assign ranks to matching lanes.

14. Final Cheat Sheet

Item	Recall
Core definition	Operations directly over integer bits
Why it matters	Compact state, hardware formats, fast masks, GPU lane collectives
Most asked	Set/test/clear, popcount, power of two, XOR, shifts
Main comparison	Bitwise vs logical; signed vs unsigned
One-line answer	"Use unsigned masks to select or modify bits, with explicit width and shift bounds."

Multithreading Basics

1. Overview

Multithreading means multiple threads of execution exist within one process and usually share its address space. Threads can improve throughput, responsiveness, and hardware utilization, but shared mutable state introduces races, ordering problems, and deadlocks.

Threads power browsers, servers, runtimes, operating systems, and CPU sides of GPU applications. GPU programming generalizes parallel execution to thousands of lightweight threads, so synchronization scope and memory visibility are foundational. Interviewers ask about threads to see whether you can reason about nondeterminism—not just name synchronization primitives.

2. Core Idea

Imagine several cooks sharing one kitchen. Work finishes faster if tasks are independent. If two cooks update the same order sheet without coordination, an update can be lost. Locks are like taking the only pen; condition variables are like ringing a bell when ingredients arrive; atomics are specialized operations on the sheet that cannot be torn into interleaving steps.

```
std::mutex m;  
int total = 0;  
  
void add(int x) {  
    std::lock_guard lock(m);  
    total += x;  
}
```

The lock establishes mutual exclusion and synchronization: only one thread changes `total` at a time, and properly synchronized threads observe preceding writes.

3. Important Subtopics

Process vs thread

A process normally owns an address space and resources; threads share those process resources but have their own stack, registers, and instruction position. Threads are cheaper to communicate between, but isolation is weaker.

Concurrency vs parallelism

Concurrency is dealing with multiple tasks whose execution overlaps in time; parallelism is executing work simultaneously. One CPU core can run concurrent threads by interleaving them; multiple cores can run them in parallel.

Race condition and data race

A race condition is any correctness dependence on timing. A C++ data race occurs when conflicting accesses to the same memory happen concurrently, at least one is a write, and no happens-before relation orders them. A C++ data race is undefined behavior.

Mutexes and lock management

A mutex provides exclusive ownership. Use RAII wrappers such as `std::lock_guard` or `std::unique_lock`, not manual lock/unlock pairs. Keep critical sections small but preserve invariants. Interviewers often ask how deadlock can arise when multiple locks are acquired inconsistently.

Atomics and memory ordering

`std::atomic<T>` provides indivisible operations and ordering guarantees. Sequential consistency is easiest to reason about; weaker acquire/release/relaxed orderings can reduce constraints but require a precise proof. Atomicity alone does not make a multi-step invariant atomic.

Condition variables

A condition variable lets threads sleep until state may have changed. Always wait with a predicate because wakeups can be spurious and another thread may consume the condition first.

```
std::unique_lock lock(m);
cv.wait(lock, [&] { return !queue.empty() || stopped; });
```

Deadlock, livelock, and starvation

Deadlock is a cycle of waiting; livelock is active response without progress; starvation is indefinite lack of access. Prevent deadlock through a global lock order, `std::scoped_lock`, or avoiding hold-and-wait.

Thread pools and task parallelism

Creating a thread per tiny job is expensive. A pool reuses workers and queues tasks. For GPU workloads, CPU threads commonly prepare data, enqueue asynchronous operations, and overlap transfer with computation.

GPU thread hierarchy and synchronization scope

GPU threads are grouped into warps/wavefronts and blocks/workgroups. A block barrier synchronizes participating threads in that block, not the entire grid. Host synchronization or separate kernel launches are common grid-wide phase boundaries.

4. Real-World Example

A backend server uses a bounded worker pool. Producer threads enqueue requests; workers wait on a condition variable, pop under a mutex, then process outside the lock. In a GPU application, one CPU thread may enqueue operations on several streams so transfers overlap with kernels. Correctness requires buffers not be overwritten until the stream records completion.

5. Diagrams / Mental Models

Process

```
|— shared code / heap / file handles
|— Thread A: registers + stack
|— Thread B: registers + stack
|— Thread C: registers + stack
```

happens-before:

```
Producer writes data -> release/store or unlock
                    -> acquire/load or lock -> Consumer reads data
```

Problem	Symptom	Typical control
Data race	Undefined behavior	Mutex or atomic protocol
Deadlock	Threads wait forever	Lock order / combined acquisition
Livelock	Threads run but make no progress	Backoff / protocol change
Starvation	One thread rarely/never proceeds	Fairness / bounded work

6. Common Interview Questions

1. **Process vs thread?** Processes have separate address spaces by default; threads share a process address space but have separate execution state. **Expected:** isolation and communication cost. **Mistake:** saying threads share stacks.
2. **Concurrency vs parallelism?** Concurrency is overlapping progress; parallelism is simultaneous execution. **Expected:** single-core example. **Mistake:** using them as exact synonyms.
3. **What is a data race?** Unsynchronized conflicting memory accesses with at least one write. **Expected:** undefined behavior in C++. **Mistake:** calling every nondeterministic result a formal data race.
4. **Mutex vs semaphore?** Mutex represents exclusive ownership; semaphore is a count of permits and need not be released by the acquiring thread. **Expected:** binary semaphore is still not necessarily ownership-based. **Mistake:** "a mutex is just a semaphore initialized to one."
5. **What is deadlock?** Threads wait in a cycle for resources. **Expected:** mutual exclusion, hold-and-wait, no preemption, circular wait. **Mistake:** confusing it with a slow lock.
6. **How do you prevent deadlock with two mutexes?** Acquire consistently or use `std::scoped_lock` to acquire together. **Expected:** RAII. **Mistake:** retry loops without progress guarantees.
7. **Why loop around condition-variable wait?** Spurious wakeups and changed predicates. **Expected:** check shared state while holding its mutex. **Mistake:** treating notification as stored state.
8. **Atomic vs mutex?** Atomic suits individual supported operations/protocols; mutex protects compound invariants and arbitrary state. **Expected:** lock-free is not automatically faster. **Mistake:** replacing every integer with atomic.
9. **What is a context switch?** Save one execution context and restore another. **Expected:** scheduler and cache/TLB costs. **Mistake:** saying only processes context-switch.
10. **What is false sharing?** Threads modify independent variables on the same cache line, causing coherence traffic. **Expected:** padding/alignment or ownership partitioning. **Mistake:** calling any shared read false sharing.
11. **What does `join` do?** Waits for a thread to finish and forms synchronization with its completion. **Expected:** joinable lifetime. **Mistake:** assuming destroying a joinable `std::thread` joins automatically—it terminates.
12. **Why can a GPU block barrier deadlock?** If only some threads reach a barrier that requires all participating block threads. **Expected:** uniform control flow around barriers. **Mistake:** assuming a barrier synchronizes other blocks.

7. Deep-Dive Questions

1. **Explain happens-before.** It is the formal ordering relation that makes side effects visible and prevents a data race. Program order plus synchronization edges such as unlock-to-lock or release-to-acquire can establish it.
2. **What is the ABA problem?** A value changes $A \rightarrow B \rightarrow A$, so compare-exchange sees A and misses intervening change. Tagged/versioned pointers or reclamation schemes can address it.
3. **Why is lock-free memory reclamation hard?** A removed node cannot be freed while another thread may still read it. Hazard pointers, epochs, or RCU-style schemes coordinate safe reclamation.
4. **How do CPU and GPU synchronization differ?** CPU mutexes coordinate a small number of heavyweight threads; GPU barriers/atomics have limited scopes and must respect execution grouping. A block barrier cannot generally solve grid-wide dependencies.
5. **When is relaxed atomic ordering enough?** For a standalone counter/statistic where the atomic value need not publish or order other memory. Atomicity remains, but no cross-object ordering is provided.

8. Comparison Tables

Feature	Process	Thread
Address space	Usually isolated	Shared within process
Stack/registers	Own	Own
Communication	IPC	Shared memory/direct calls
Failure isolation	Stronger	Weaker
Creation/switch cost	Usually higher	Usually lower

Primitive	Models	Sleeps?	Best use
Mutex	Exclusive ownership	Under contention, usually	Compound shared invariant
Semaphore	Permit count	Usually	Resource slots/signaling
Atomic	Indivisible operation/order	No lock wait by API	Small proven protocols/counters
Condition variable	Wait for state change	Yes	Queues and predicates
Barrier	Phase rendezvous	Implementation-dependent	All participants finish a phase

9. Common Mistakes

- Protecting writes but not reads of the same non-atomic object.
- Assuming `volatile` makes code thread-safe.
- Holding a mutex during slow I/O or callbacks without need.
- Waiting on a condition variable without a predicate.
- Capturing references whose objects die before a thread completes.
- Detaching threads to avoid lifecycle design.
- Assuming different variables cannot false-share.
- Placing a GPU barrier inside divergent control flow.

10. Edge Cases / Special Cases

- Two reads never conflict; read/write and write/write do.
- Different mutexes do not protect the same invariant unless the protocol connects them.
- Recursive mutexes can hide broken ownership design and do not solve cross-thread cycles.
- Atomic operations can be non-lock-free on some types/platforms.
- Oversubscription can reduce performance through switching and cache disruption.
- Exceptions escaping a thread function terminate the program unless caught inside it.
- GPU atomic ordering and scope must match the communicating threads and memory space.

11. How to Explain in Interview

“Threads share process memory but have independent execution state. Parallel speedup comes from partitioning independent work; correctness comes from establishing happens-before relations around shared mutable state using mutexes, atomics, or higher-level queues. On GPUs I also state the synchronization scope—warp, block, device, or host.”

12. Quick Revision Notes

- C++ data race = undefined behavior.
- Mutex protects invariants; atomic protects atomic operations/protocols.
- Wait for predicates, not notifications.
- Avoid deadlock with consistent lock order or combined locking.
- `join` completes thread lifetime safely.
- False sharing is cache-line contention without logical sharing.
- GPU barriers have limited scope.

13. Practice Tasks

1. Implement a bounded producer-consumer queue with one mutex and condition variables.
2. Create a counter race, run ThreadSanitizer, then fix it.
3. Demonstrate deadlock with reversed lock order and repair it using `std::scoped_lock`.
4. Compare mutex and atomic counters under different contention levels.
5. Measure false sharing with adjacent counters, then separate them by cache-line alignment.
6. Write a GPU block reduction and justify every barrier.

14. Final Cheat Sheet

Item	Recall
Core definition	Multiple execution threads sharing process resources
Why it matters	Throughput and CPU/GPU orchestration, with nondeterministic hazards
Most asked	Race, mutex/atomic, condition variable, deadlock, false sharing
Main comparison	Process vs thread; mutex vs semaphore vs atomic
One-line answer	“Partition work first; synchronize only shared state, with explicit ordering and scope.”

CPU Cache Hierarchy

1. Overview

A CPU cache is a small, fast memory that keeps copies of recently or nearby used data so cores avoid waiting for slower main memory. Modern systems typically have private L1 caches, larger L2 caches, and a larger shared last-level cache before DRAM.

Caches matter because processor execution is much faster than DRAM access. Data layout and access order can dominate performance in databases, browsers, servers, numerical code, and CPU preparation for GPU

workloads. Interviewers ask about locality, cache lines, misses, coherence, and false sharing to connect algorithms with real hardware.

2. Core Idea

Think of a desk, cabinet, room archive, and distant warehouse. Items on the desk are fastest to reach but space is tiny. When the CPU asks for one byte, hardware usually fetches an entire cache line containing nearby bytes. Code is fast when it reuses that line before eviction.

```
// Row-major matrix: good locality
for (int r = 0; r < rows; ++r)
    for (int c = 0; c < cols; ++c)
        sum += a[r * cols + c];
```

Adjacent inner-loop accesses share cache lines. Swapping the loops may jump by `cols` elements each time and cause many misses.

3. Important Subtopics

L1, L2, LLC, and DRAM

L1 is smallest and fastest, often split into instruction and data caches. L2 is larger/slower, and the last-level cache (often L3) may be shared by cores. Exact organization varies by CPU, so state principles rather than memorizing universal sizes.

Cache lines and locality

Data moves in fixed-size cache lines, commonly 64 bytes on desktop CPUs but not guaranteed. Temporal locality means reusing data soon; spatial locality means accessing nearby addresses.

Cache mapping and associativity

An address maps to a cache set; an N-way set-associative cache can hold N matching lines in that set. Too many active addresses mapping to one set create conflict misses even when total working data seems small.

Hit and miss types

- Compulsory miss: first access to a line.
- Capacity miss: working set exceeds the cache.
- Conflict miss: mapping contention evicts a line.
- Coherence miss: another core's write invalidates a line.

Replacement and write policies

Caches approximate replacement policies such as LRU. Write-back caches defer DRAM updates until eviction; write-through caches propagate writes immediately. Write-allocate loads a line before writing it; non-temporal stores may avoid cache pollution for streaming output.

Prefetching

Hardware detects regular access patterns; software prefetch can help only when timing is predictable and work hides latency. Excess prefetch wastes bandwidth and cache capacity. Measure before adding it.

Cache coherence

Coherence protocols ensure cores converge on a consistent value for each cache line, commonly via states conceptually like Modified, Exclusive, Shared, Invalid. Coherence does not define all program ordering; the language memory model still requires synchronization.

False sharing

Independent variables on one line still share coherence state. Repeated writes by different cores bounce ownership of the line. Partitioning by thread or aligning hot counters can help.

Relation to GPU memory

GPUs also use caches, but throughput relies heavily on coalescing, massive concurrency, and explicit shared memory. CPU cache-friendly tiling and GPU shared-memory tiling share the idea of reusing a small working set near compute units.

4. Real-World Example

Matrix multiplication reuses rows and columns many times. A naive loop can repeatedly pull data from lower cache levels. Blocking divides matrices into tiles sized for cache, performs many operations per loaded tile, then moves on. The same principle appears in GPU tiled kernels, where blocks cooperatively load tiles into shared memory.

Databases similarly use cache-conscious B-trees and columnar scans to maximize useful work per fetched line.

5. Diagrams / Mental Models

```
Core -> L1 (tiny, fastest) -> L2 -> shared LLC/L3 -> DRAM (large, slow)
      hit: continue          miss: request next level
```

```
Cache line with false sharing:
[counter for thread 0 | counter for thread 1 | unused bytes]
core 0 write <----- line ownership -----> core 1 write
```

Locality	Meaning	Example
Temporal	Reuse the same data soon	Accumulator/tile reused in inner loop
Spatial	Use neighboring data	Sequential array scan
Poor locality	Large or irregular jumps	Pointer chasing/random indices

6. Common Interview Questions

1. **Why do CPUs need caches?** CPU execution outpaces DRAM latency/bandwidth, so caches exploit locality. **Expected:** hierarchy tradeoff. **Mistake:** saying cache increases DRAM speed.

2. **What is a cache line?** Minimum granularity commonly transferred and coherently tracked. **Expected:** nearby bytes arrive together. **Mistake:** assuming universal 64-byte size.
3. **Temporal vs spatial locality?** Reuse soon versus nearby addresses soon. **Expected:** concrete examples. **Mistake:** confusing locality with physical CPU cores.
4. **What happens on a cache miss?** Lower levels are checked; a line is fetched and may evict another. **Expected:** miss penalty and possible write-back. **Mistake:** claiming every miss goes directly to DRAM.
5. **Why is row-major traversal faster?** Sequential access uses every fetched line and prefetches well. **Expected:** C/C++ layout. **Mistake:** treating it as an algorithmic Big-O difference.
6. **What is false sharing?** Independent writes share a cache line and trigger coherence traffic. **Expected:** line granularity. **Mistake:** defining it as two threads writing the same variable.
7. **Cache coherence vs consistency?** Coherence concerns ordering/visibility for individual locations; consistency/memory model constrains ordering across operations. **Expected:** synchronization still required. **Mistake:** "coherent means data races are safe."
8. **What is associativity?** Number of lines a set can hold. **Expected:** direct-mapped = one-way, fully associative = any line. **Mistake:** confusing ways with cache levels.
9. **What are compulsory, capacity, and conflict misses?** First-touch, too-large working set, and mapping collision. **Expected:** remedies differ. **Mistake:** labeling every miss capacity.
10. **Why can linked lists be slower than arrays?** Pointer chasing has weak spatial locality and dependent loads. **Expected:** allocator/layout and prefetch limits. **Mistake:** relying only on Big-O.
11. **What is cache blocking?** Reorder computation to reuse tiles while resident in cache. **Expected:** matrix multiplication example. **Mistake:** making blocks without considering working-set size.
12. **How does false sharing affect atomics?** Even separate atomics on one line can contend through coherence. **Expected:** placement matters. **Mistake:** assuming lock-free means contention-free.

7. Deep-Dive Questions

1. **Derive average memory access time.** For one cache, $\text{hit_time} + \text{miss_rate} \times \text{miss_penalty}$; multilevel models recursively expand the penalty. It is an average and can hide overlap from out-of-order execution.
2. **Why can power-of-two strides be bad?** Address bits used for set indexing may repeat, mapping many lines to the same few sets and causing conflict misses. Padding can change the mapping.
3. **How do write-back and write-allocate interact?** A store miss may fetch the line, modify it in cache, and mark it dirty; eviction later writes the whole line back. Streaming stores may avoid this read-for-ownership.
4. **Why doesn't a cache flush fix a C++ data race?** Language-level undefined behavior and compiler reordering remain. Correct atomics/locks establish the required abstract-machine ordering.
5. **How would you choose a matrix tile size?** Account for all simultaneously live tiles, element size, cache capacity/associativity, vector width, and threading; then benchmark because effective cache availability varies.

8. Comparison Tables

Level	Relative size	Relative latency	Typical sharing
Registers	Tiny	Lowest	One hardware thread
L1	Small	Very low	Usually one core

Level	Relative size	Relative latency	Typical sharing
L2	Medium	Low/moderate	Often one core or small cluster
LLC/L3	Large	Higher	Often multiple cores
DRAM	Very large	Highest	System memory
Miss type	Cause	Typical response	
Compulsory	First touch	Prefetch/sequential access	
Capacity	Working set too large	Blocking/smaller working set	
Conflict	Too many lines map to same set	Padding/layout change	
Coherence	Other core invalidates line	Reduce shared writes/false sharing	

9. Common Mistakes

- Treating cache sizes and line widths as universal constants.
- Optimizing only Big-O while ignoring memory access patterns.
- Confusing cache locality with virtual-memory locality.
- Assuming sequential traversal is always faster without considering work performed.
- Padding every struct preemptively instead of measuring contention.
- Believing `volatile` controls cache coherence.
- Ignoring NUMA placement in large multi-socket systems.
- Applying CPU cache advice directly to GPUs without considering coalescing and occupancy.

10. Edge Cases / Special Cases

- Cache lines crossing page boundaries can require multiple translations/accesses.
- Inclusive, exclusive, and non-inclusive cache policies differ across processors.
- Simultaneous multithreading may share core cache resources.
- Prefetchers struggle with pointer chains and unpredictable indices.
- A smaller data type may improve cache density even if arithmetic converts it.
- Memory-mapped I/O requires special ordering/cache rules; ordinary memory assumptions may fail.
- Non-uniform memory access means “DRAM latency” depends on which socket owns the pages.

11. How to Explain in Interview

“CPU caches bridge the latency gap between cores and DRAM by moving data in lines and exploiting temporal and spatial locality. I optimize the working set and traversal order first, then consider blocking, contention, and false sharing. Coherence moves lines correctly, but program synchronization still defines legal visibility.”

12. Quick Revision Notes

- Hierarchy trades capacity for latency.
- Transfers and coherence operate at cache-line granularity.
- Sequential arrays usually beat pointer chasing.
- Three classic misses: compulsory, capacity, conflict; coherence adds another practical class.

- Blocking increases reuse.
- False sharing = different data, same line, multiple writers.
- Cache coherence does not remove data races.

13. Practice Tasks

1. Benchmark row-major versus column-major matrix traversal.
2. Vary an array's working-set size and plot latency changes.
3. Benchmark pointer chasing against a contiguous scan.
4. Demonstrate false sharing with per-thread counters and fix their placement.
5. Implement blocked CPU matrix multiplication and tune one tile dimension at a time.
6. Use a profiler to record cache-miss counters for the experiments.

14. Final Cheat Sheet

Item	Recall
Core definition	Small fast memory holding copies of recently/nearby used lines
Why it matters	Memory behavior often limits real performance
Most asked	Locality, cache lines, miss types, coherence, false sharing
Main comparison	L1/L2/LLC/DRAM; temporal/spatial locality
One-line answer	"Make the hot working set small, contiguous, reusable, and minimally shared by writers."

Virtual Memory

1. Overview

Virtual memory gives each process an address space whose virtual addresses are translated to physical memory or other backing storage. It provides isolation, flexible placement, sparse address spaces, sharing, protection, and demand paging.

Operating systems, databases, browsers, memory-mapped files, and GPU unified/virtual memory depend on it. Interviewers ask about pages, page tables, TLBs, faults, swapping, and translation because these connect application behavior to OS and hardware performance.

2. Core Idea

Think of a hotel room number as a virtual address. Guests use stable room numbers; the front desk's mapping says which physical room or storage location currently holds the contents. The page offset is the position inside the room and does not change during translation.

```
virtual address = [ virtual page number | page offset ]
                  | page table/TLB
```

$$\text{physical address} = [\text{physical frame number} \mid \text{same offset}]$$

The CPU first checks a translation cache called the TLB. On a TLB miss, hardware/software walks page tables. If the mapping is absent or violates permissions, a page fault transfers control to the OS.

3. Important Subtopics

Pages and frames

Virtual address spaces are divided into fixed-size pages; physical memory into equal-size frames. A page-table entry maps a page to a frame and stores permission/presence metadata. Fixed-size units avoid external fragmentation but can waste space inside the final page.

Page tables

Because a flat page table would be huge, systems use multilevel/radix tables and allocate lower levels only for used address regions. The exact number of levels and address bits is architecture-specific.

Translation Lookaside Buffer

The TLB caches recent translations. A TLB hit avoids a page-table walk; a miss is not necessarily a page fault. Large pages increase the memory covered per TLB entry but increase allocation/fragmentation concerns.

Page faults

A minor/soft fault can be resolved without disk I/O, such as mapping an already resident shared page or allocating a zero page. A major/hard fault requires storage I/O and is much slower. An invalid access may terminate the process rather than be resolved.

Demand paging and swapping

Pages can be populated only when touched. Under pressure, the OS may evict pages; dirty anonymous pages may go to swap, while clean file-backed pages can often be dropped and reread. Excessive faulting is thrashing.

Protection and isolation

Page-table flags control read, write, execute, privilege, and other access properties. Separate mappings isolate processes; shared mappings deliberately expose the same frames.

Copy-on-write

After `fork`, parent and child can share read-only physical pages. A write triggers a fault that creates a private copy. This makes process creation cheap when most pages are not immediately modified.

Memory-mapped files

`mmap`-style APIs map file contents into an address range. Pages are loaded on demand and the OS page cache can unify file I/O and memory access. Correct durability still requires explicit semantics such as syncing and

filesystem guarantees.

GPU virtual and unified memory

Modern GPU systems can present unified virtual addressing, while managed/unified memory can migrate pages between CPU and GPU. Oversubscription becomes possible, but page faults and migration across PCIe/interconnects can dominate runtime. Access-pattern-aware prefetch/advice may help.

4. Real-World Example

A database memory-maps a large index file. Its address space can represent the whole file even if only a small working set is resident. Access to a missing page faults it into RAM; frequently used index pages remain cached. Random access across a huge mapping may overload the TLB and page cache.

A GPU unified-memory application behaves similarly: first GPU touch may fault and migrate pages to the device, so a kernel can stall despite simple source code.

5. Diagrams / Mental Models

```

CPU virtual address
  |
  v
  TLB hit? --yes--> physical frame + offset
  |
  no
  v
  page-table walk --> valid mapping? --yes--> fill TLB
                      |
                      no
                      v
                    page fault -> OS resolves or rejects
  
```

Event	Meaning
TLB hit	Translation cached
TLB miss	Page-table lookup required
Minor fault	OS resolves without storage I/O
Major fault	Backing-store I/O required
Protection fault	Access violates mapping permissions

6. Common Interview Questions

1. **What is virtual memory?** Per-process virtual addresses translated to physical frames/backing storage.
Expected: isolation and abstraction, not just "extra RAM." **Mistake:** equating it solely with swap.
2. **What is a page?** Fixed-size virtual-memory unit; a frame is its physical counterpart. **Expected:** offset retained in translation. **Mistake:** using page and frame without distinction.

3. **What is a page table?** Mapping plus metadata from virtual pages to physical frames. **Expected:** per-address-space roots and multilevel structure. **Mistake:** assuming one global flat table.
4. **What is a TLB?** Hardware cache of recent translations. **Expected:** TLB miss vs page fault. **Mistake:** saying a TLB miss reads from disk.
5. **What happens on a page fault?** Hardware traps; OS validates the access, obtains/maps data or reports an invalid access, updates tables, and resumes if resolvable. **Expected:** multiple causes. **Mistake:** saying every fault is an error.
6. **Internal vs external fragmentation?** Fixed pages waste space inside allocations; variable contiguous allocation creates unusable gaps. **Expected:** paging largely avoids external fragmentation. **Mistake:** claiming paging has no fragmentation.
7. **What is demand paging?** Populate a mapping only when accessed. **Expected:** startup/memory benefits and first-touch cost. **Mistake:** assuming reserved virtual space consumes equal physical RAM immediately.
8. **What is copy-on-write?** Share pages until a writer faults and receives a private copy. **Expected:** fork example. **Mistake:** saying the entire process is copied at fork.
9. **Why use huge pages?** More address coverage per TLB entry and shorter walks. **Expected:** fragmentation and allocation tradeoffs. **Mistake:** assuming they always improve performance.
10. **What is thrashing?** The working set exceeds available memory, causing continual eviction/faulting and little useful progress. **Expected:** locality/working-set fix. **Mistake:** describing any swapping as thrashing.
11. **Segmentation vs paging?** Variable logical regions versus fixed-size pages; modern general-purpose systems mainly expose paging while segments may have limited roles. **Expected:** fragmentation differences. **Mistake:** assuming every architecture uses classic segmentation.
12. **What can make unified GPU memory slow?** On-demand page faults, migration, ping-pong, oversubscription, and poor locality. **Expected:** prefetch/placement/access pattern. **Mistake:** assuming "unified" means physically one equally fast memory.

7. Deep-Dive Questions

1. **How does a multilevel page table save memory?** It allocates lower-level tables only for populated portions of a sparse virtual address space, trading memory savings for extra walk accesses.
2. **What is an address-space identifier (ASID/PCID)?** A tag on TLB entries distinguishing address spaces, reducing the need to flush all translations on context switches.
3. **How does NUMA interact with first touch?** Physical pages are often allocated near the CPU that first writes them. Poor initialization placement can make later accesses remote.
4. **Why can pinned memory not be overused?** It cannot be freely paged out, reducing OS flexibility and available pageable memory. It is valuable for DMA but a constrained resource.
5. **How does CPU-GPU page ping-pong occur?** Alternating writes by CPU and GPU force ownership/migration back and forth. Partition ownership, batch phases, or place/prefetch data deliberately.

8. Comparison Tables

Feature	Paging	Segmentation
Unit size	Fixed	Variable/logical segment
Programmer visibility	Usually hidden	May reflect code/data/stack

Feature	Paging	Segmentation
Fragmentation	Internal	External
Placement	Any free frame	Needs suitable contiguous region

Feature	Virtual address	Physical address
Seen by normal process	Yes	Usually no
Stable across relocation	Yes	Mapping may change
Contains	Virtual page + offset	Frame + offset

GPU memory mode	Convenience	Performance responsibility
Explicit device allocation/copy	More manual	Predictable transfers/placement
Unified/managed memory	Easier pointer model	Migration/fault behavior must be managed
Pinned host memory	DMA/asynchronous transfer friendly	Scarce; allocation/access tradeoffs

9. Common Mistakes

- Defining virtual memory as “disk used as RAM” only.
- Confusing TLB miss, page fault, and cache miss.
- Assuming allocation/reservation immediately commits and touches every physical page.
- Treating a segmentation fault as proof that segmentation caused it.
- Ignoring page size when analyzing random-access performance.
- Assuming memory-mapped writes are instantly durable.
- Overusing pinned or huge-page memory.
- Expecting unified GPU memory to eliminate transfer costs.

10. Edge Cases / Special Cases

- Accessing a valid virtual address can still fault normally on first touch.
- Stack growth may be implemented by faults within guarded limits.
- Guard pages intentionally have no access permission.
- Shared pages can map at different virtual addresses in different processes.
- Transparent huge pages may promote/split mappings automatically, causing variable behavior.
- Page-table walks themselves access memory and may be cached.
- GPU page sizes, migration granularity, and fault support vary by architecture and runtime.

11. How to Explain in Interview

“Virtual memory maps process-visible pages to physical frames, giving isolation, protection, sharing, and flexible allocation. The TLB caches translations; a miss causes a page-table walk, while a page fault invokes the OS because the mapping is absent or disallowed. For GPUs, unified addressing simplifies pointers but migration and placement remain performance concerns.”

12. Quick Revision Notes

- Virtual page number is translated; offset stays unchanged.
- TLB miss is not a page fault.
- Page faults can be normal, minor, major, or invalid/protection-related.
- Paging: fixed size, internal fragmentation.
- COW shares until write.
- Huge pages improve TLB reach but cost flexibility.
- Unified memory simplifies access, not physical movement.

13. Practice Tasks

1. Split sample virtual addresses into page number and offset for several page sizes.
2. Draw a two-level page-table walk and count potential memory accesses.
3. Allocate a large region and measure reserve versus first-touch behavior.
4. Compare sequential and random page access and observe fault/TLB counters.
5. Explain **fork** memory behavior using copy-on-write.
6. Compare explicit GPU copies with unified-memory first-touch and prefetch behavior.

14. Final Cheat Sheet

Item	Recall
Core definition	Translation from protected virtual pages to physical frames/backing storage
Why it matters	Isolation, flexible allocation, sharing, and CPU/GPU migration behavior
Most asked	Page table, TLB, page fault, COW, huge pages, thrashing
Main comparison	Paging vs segmentation; TLB miss vs page fault
One-line answer	"Virtual memory separates addresses from placement; TLBs make translation fast and faults let the OS resolve missing mappings."

Basic Linear Algebra: Vectors, Matrices, and Matrix Multiplication

1. Overview

A vector is an ordered list of scalars; a matrix is a rectangular grid of scalars that can represent data or a linear transformation. Matrix multiplication composes transformations and combines rows of the left matrix with columns of the right matrix.

Linear algebra underlies graphics, machine learning, simulations, signal processing, robotics, and scientific computing—the workloads GPUs are designed to accelerate. Interviewers ask it to test dimension reasoning, indexing, algorithmic complexity, numerical awareness, and the mapping of mathematics onto parallel hardware.

2. Core Idea

A vector can describe a direction, a point, or a feature list. A matrix can transform one vector into another. Think of a matrix as a machine: each output coordinate is a weighted combination of input coordinates.

For

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad x = \begin{bmatrix} 5 \\ 7 \end{bmatrix} \quad Ax = \begin{bmatrix} 1*5 + 2*7 \\ 3*5 + 4*7 \end{bmatrix} = \begin{bmatrix} 19 \\ 43 \end{bmatrix}$$

Matrix multiplication repeats this dot-product idea. If A is $m \times k$ and B is $k \times n$, then $C = AB$ is $m \times n$, with

$$C[i,j] = \text{sum over } p=0..k-1 \text{ of } A[i,p] * B[p,j]$$

Step by step: select row i from A ; select column j from B ; multiply corresponding entries; add them; store the scalar at $C[i,j]$. Every output element can be computed independently once inputs are available, which exposes massive parallelism.

3. Important Subtopics

Scalars, vectors, and dimensions

A scalar is one number. A vector in $\mathbb{R}^n$ has n components. Dimensions must agree for addition; scalar multiplication scales every component. In code, shape is part of correctness even when the type system does not express it.

Vector addition, magnitude, and normalization

Addition is component-wise. Euclidean magnitude is $\|v\|_2 = \sqrt{\sum v_i^2}$. Normalization computes $v / \|v\|$ to make unit length. A zero vector cannot be normalized by ordinary division.

Dot product

$a \cdot b = \sum a_i b_i$. It measures alignment and produces a scalar. For nonzero vectors, $a \cdot b = \|a\| \|b\| \cos \theta$. It is used for similarity, projections, lighting, and reductions on GPUs.

Matrices and shapes

An $m \times n$ matrix has m rows and n columns. Addition requires equal shapes. Transpose swaps axes: if A is $m \times n$, A^T is $n \times m$ and $(A^T)[j,i] = A[i,j]$.

Matrix multiplication

AB exists when the inner dimensions match. The result uses the outer dimensions. It is associative, generally not commutative, and distributes over addition.

Identity, diagonal, and inverse

The identity matrix $\mathbf{I}$ leaves vectors unchanged. A diagonal matrix scales coordinates independently. An inverse $\mathbf{A}^{-1}$ satisfies $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$, but exists only for nonsingular square matrices. In numerical code, solve $\mathbf{Ax}=\mathbf{b}$ directly rather than explicitly computing the inverse when possible.

Row-major and column-major storage

Storage order is not the mathematical matrix. Row-major makes row elements contiguous; column-major makes column elements contiguous. Libraries specify layouts and leading dimensions/strides. A transpose flag may reinterpret access without physically moving data.

Naive multiplication complexity

For square $n \times n$ matrices, the standard algorithm uses $O(n^3)$ arithmetic and $O(n^2)$ output storage. For $m \times k$ times $k \times n$, work is $O(mkn)$. Each output performs k multiply-add terms.

Tiling on GPUs

Naive GPU multiplication repeatedly loads the same input values from global memory. A thread block loads tiles of $\mathbf{A}$ and $\mathbf{B}$ into shared memory, synchronizes, performs many multiply-accumulates, and advances through the inner dimension. Tiling increases arithmetic intensity.

Floating-point accuracy

Floating-point addition is not associative; changing reduction order can change low bits. Accumulating many products can lose precision. Fused multiply-add performs $\mathbf{a}*\mathbf{b}+\mathbf{c}$ with one final rounding and is common on GPUs.

BLAS and GEMM

BLAS defines standard vector/matrix operations. GEMM computes a form such as $\mathbf{C} = \alpha\mathbf{AB} + \beta\mathbf{C}$ and is heavily optimized for CPU and GPU hardware. Production code generally calls tuned libraries such as vendor BLAS rather than writing a general GEMM from scratch.

4. Real-World Example

In a neural-network layer, a batch of input vectors is a matrix $\mathbf{X}$, weights form $\mathbf{W}$, and outputs are $\mathbf{Y} = \mathbf{XW} + \mathbf{b}$. Thousands of output elements are independent dot products. GPU GEMM kernels tile $\mathbf{X}$ and $\mathbf{W}$, reuse values from shared memory/registers, and may use specialized tensor/matrix units. Correct shapes, layout, precision, and synchronization determine both correctness and speed.

Graphics uses the same idea: matrices rotate, scale, translate (with homogeneous coordinates), and project vertices.

5. Diagrams / Mental Models

$\mathbf{A} \ (m \times k)$		$\mathbf{B} \ (k \times n)$		$\mathbf{C} \ (m \times n)$
row i : $[a \ a \ a]$	$\cdot$	column j : $\begin{bmatrix} b \\ b \\ b \end{bmatrix}$	$\rightarrow$	$\mathbf{C}[i,j]$

```
GPU tiled multiplication:
global A tile -> shared memory --\
                                multiply-accumulate -> C tile
global B tile -> shared memory --/
    load, barrier, compute, barrier, advance
```

Operation	Input shapes	Output
Vector addition	n, n	vector n
Dot product	n, n	scalar
Matrix-vector	$m \times n, n$	vector m
Matrix-matrix	$m \times k, k \times n$	matrix $m \times n$
Transpose	$m \times n$	matrix $n \times m$

6. Common Interview Questions

1. **What is a vector?** An ordered collection of scalars that can represent a point, direction, or feature. **Expected:** dimension and operations. **Mistake:** confusing `std::vector` ownership with the mathematical concept.
2. **What is a dot product?** Sum of pairwise products, producing a scalar. **Expected:** equal dimensions and geometric meaning. **Mistake:** returning a vector.
3. **When can matrices be multiplied?** Columns of the left matrix must equal rows of the right. **Expected:** result shape uses outer dimensions. **Mistake:** requiring the two matrices to have identical shapes.
4. **Why is matrix multiplication not commutative?** AB and BA compose transformations in different orders and may have different shapes. **Expected:** small counterexample. **Mistake:** saying it is never equal—special cases commute.
5. **Complexity of naive matrix multiplication?** $O(mkn)$; square case $O(n^3)$. **Expected:** output storage $O(mn)$. **Mistake:** saying $O(n^2)$ because there are n^2 outputs, ignoring each dot product.
6. **What does transpose do?** Swaps row and column indices. **Expected:** $(AB)^T = B^T A^T$. **Mistake:** merely reversing a flat array without considering layout.
7. **Row-major vs column-major?** Different contiguous storage orders for the same mathematical matrix. **Expected:** indexing/stride effects. **Mistake:** claiming they change matrix values.
8. **Why are GPUs good at matrix multiplication?** Many independent outputs, high data parallelism, high bandwidth, and tile reuse; specialized units may accelerate multiply-accumulate. **Expected:** parallelism plus locality. **Mistake:** saying only “more cores.”
9. **What is tiling?** Process submatrices that fit fast local storage so loaded values are reused. **Expected:** synchronization and boundary handling. **Mistake:** assuming any tile size is valid or fastest.
10. **What is the identity matrix?** Square matrix with ones on the diagonal and zeros elsewhere; $AI=IA=A$ where dimensions fit. **Expected:** transformation interpretation. **Mistake:** calling an all-ones matrix identity.
11. **When does an inverse exist?** For a square nonsingular matrix, equivalently full rank/determinant nonzero in exact arithmetic. **Expected:** numerical conditioning caveat. **Mistake:** assuming every square matrix is invertible.

12. **Why can parallel reductions differ numerically?** Floating-point addition is not associative, so different grouping changes rounding. **Expected:** tolerance and stable accumulation. **Mistake:** labeling any small difference a race.

7. Deep-Dive Questions

1. **What is arithmetic intensity, and why does GEMM have high potential intensity?** It is operations per byte moved from a memory level. Tiling reuses each loaded matrix value across many multiply-adds, so computation grows faster than required external data movement.
2. **How does a tiled GPU GEMM handle dimensions not divisible by tile size?** Threads conditionally load valid elements and substitute zero for out-of-range input; output stores are guarded. All required threads must still participate safely in block barriers.
3. **Why is explicitly computing $A^{-1}b$ usually inferior to solving $Ax=b$?** Inversion does extra work, uses more memory, and can amplify numerical error. Factorization plus solves is more stable and efficient.
4. **What are leading dimensions?** Physical strides between consecutive rows or columns in a storage layout. They allow submatrices and padded allocations; confusing logical shape with stride causes wrong indexing.
5. **How do precision formats affect GPU GEMM?** Lower precision reduces bandwidth/storage and can use faster tensor units, but range and rounding worsen. Mixed precision often multiplies low-precision inputs and accumulates in higher precision, sometimes with loss scaling or refinement.

8. Comparison Tables

Concept	Vector	Matrix
Shape	n (or $n \times 1$ by convention)	$m \times n$
Typical meaning	Point/direction/features	Dataset/linear transformation
Core operation	Dot product	Matrix-vector/matrix-matrix product
Layout	Address of (r, c)	Contiguous direction
Row-major	$\text{base} + r * \text{stride} + c$	Across columns in a row
Column-major	$\text{base} + c * \text{stride} + r$	Across rows in a column
Approach	Memory behavior	Use
Naive GEMM	Repeated global loads, limited reuse	Teaching/small baseline
Tiled GEMM	Reuse through cache/shared memory	Optimized kernels
Tuned BLAS GEMM	Architecture-specific packing/tiling/vector units	Production default
Precision	Benefit	Risk
FP64	Range/precision	Lower throughput on many GPUs, more bandwidth
FP32	Common balance	Accumulation error for difficult problems
FP16/BF16	High throughput, compact	Lower precision/range; often mixed accumulation

Precision	Benefit	Risk
Integer quantized	Very compact/fast on supported units	Scale/zero-point and accuracy complexity

9. Common Mistakes

- Multiplying matrices with incompatible inner dimensions.
- Reporting the wrong result shape.
- Assuming matrix multiplication is element-wise or commutative.
- Confusing mathematical transpose with a storage-layout label.
- Using `row * rows + col` instead of `row * columns + col` for row-major data.
- Normalizing the zero vector.
- Comparing floating-point results for exact equality.
- Writing a custom production GEMM when a tuned library is available.
- Adding synchronization inside divergent GPU control flow.
- Ignoring boundary tiles or leading dimensions.

10. Edge Cases / Special Cases

- Zero-sized dimensions are valid in some APIs and should perform no out-of-range access.
- `1x1` matrix multiplication resembles scalar multiplication but retains matrix shape.
- Sparse matrices need specialized formats/algorithms; dense GEMM assumptions waste work.
- Singular and near-singular matrices differ: the latter may be invertible but numerically unstable.
- NaN and infinity propagate according to floating-point rules and can defeat ordinary tolerances.
- Aliasing input and output buffers may be unsupported by BLAS/kernel contracts.
- Very small matrices may be slower through a heavyweight GPU launch than on the CPU.
- Non-deterministic scheduling can change summation order even in race-free kernels.

11. How to Explain in Interview

“A matrix represents data or a linear transformation. Multiplying an `m×k` matrix by a `k×n` matrix produces `m×n`, where each output is a row-column dot product. That exposes `mn` parallel outputs, and GPU kernels tile the shared `k` dimension so values loaded from global memory are reused many times.”

12. Quick Revision Notes

- Dot product: $\sum a_i b_i$; output is scalar.
- `m×k` times `k×n` gives `m×n`.
- Square naive GEMM: $O(n^3)$ work, $O(n^2)$ output.
- Multiplication is associative, not generally commutative.
- $(AB)^T = B^T A^T$.
- Row-major and column-major are storage choices.
- Tiling improves reuse/arithmetic intensity.
- Floating-point reduction order matters.
- Prefer tuned BLAS for production GEMM.

13. Practice Tasks

1. Compute several matrix products by hand and state shape before values.
2. Implement vector addition, dot product, transpose, and naive GEMM in C++.
3. Add shape validation and test rectangular matrices and zero dimensions.
4. Benchmark loop-order permutations for CPU GEMM and explain cache behavior.
5. Implement a boundary-safe tiled GPU GEMM and compare it with the naive kernel.
6. Compare the custom result against a BLAS implementation using absolute and relative tolerances.
7. Measure FP32 versus mixed-precision performance and numerical error.
8. Perform a roofline-style estimate: arithmetic operations divided by bytes transferred.

14. Final Cheat Sheet

Item	Recall
Core definition	Vectors are ordered scalars; matrices represent rectangular data/linear maps; multiplication is row-column dot products
Why it matters	Core workload in graphics, ML, simulation, and GPU computing
Most asked	Shape rules, dot product, complexity, layout, tiling, floating-point order
Main comparison	Row/column-major; naive/tiled/library GEMM; FP32/mixed precision
One-line answer	"GEMM computes independent row-column dot products and gets fast by reusing tiled data near the processors."